

Overall vision of SED2

A SED2 document contains two sections for defining simulation experiments: Tasks and Outputs. Tasks define how to do things, and Outputs define how to organize the results. In addition, the SED2 document contains predefined constants (so they can be centrally located and changed), styles, and global algorithm parameters.

Tasks: inputs and outputs. Each individual Task must describe what its inputs and what its outputs are. When executing the task, the inputs are used, the task is run with the input data, and outputs of the task are generated. I.e. the task stitches everything together, from inputs to outputs.

Chronological Organization: The organization of the Tasks and Outputs are that no Task nor Output may rely on input from a later Task. Thus, if someone wants to create a script to perform the operations in the SED file, they should be able to do so following the order of Tasks and then Outputs in the file. However, more sophisticated interpreters may be able to distribute Tasks across multiple parallel computers by following the connected nodes of input and output chains.

When SED defines a repeat, the task children of the repeat follow the same rules as the outermost list of tasks: they may depend on each other, and when they do, they must be listed in dependency order.

Abstraction. SED 2 is not unlike workflow languages, with one significant difference: it describes general abstract tasks, and does not rely on specific implementations. So while a workflow language might include the command `'ls -asF'`, the SED equivalent might be a 'list the contents of a directory' task where the input is a directory, the output is a list of strings, and there are a handful of parameters to indicate that all contents are to be listed, with their sizes listed, and indicate whether each is a file or a directory.

Types

Every element in a SED file can either be a value or a reference to a value. This means that anywhere in SED 2, if something is listed as requiring a 'double', it could either be an actual number (6.24), or it could be a reference to another task value that itself is a double

(`"#tasks:sim1.model["S1"]`). Values can be:

- Numbers (i.e. integers, doubles, etc.)
- Booleans
- Strings
- AnnotatedData (n-dimensional matrices of values with labeled coordinates and dimensions)
- Models
- Map ({type: type})

The AnnotatedData type is modeled after the [xarray.DataArray](#), which in turn is similar to a multi-dimensional DataFrame.

A 'Model' is two things: a set of values (or state of the model), and a set of rules for how those values change. The values in a model may be accessed by SED as if the model was a dictionary, with values accessible by using the IDs in that model. I.e. `model["S1"]` will get the current value of the element "S1" in the model. Generally, if a simulation-type Task is performed with a Model input, it will also have a Model output, where the set of rules have not changed (i.e. 'the reactions and rates of change formulas'), but the state will change (i.e. 'what you get from `model["S1"]`').

Different model languages are supported (e.g. SBML, CellML, NeuroML). The respective model language must clearly define how to access the state (internal values), i.e. via which identifier patterns. E.g. for SBML "SId" accesses the value of the object in SBML with the respective SId, e.g. the value of a parameter, a compartment size or the concentration/amount of a species (if the `hasOnlySubstanceUnits` is false/true, respectively).

A Map is a 1:1 mapping of one type to another. One example is {SId: SIdRef}, which maps an exposable SId string to an internal SIdRef string.

Special rules about numbers:

- The strings "nan", "inf" and "-inf" (and any alternative capitalizations of those strings) represent the numbers NaN, infinity, and -infinity, respectively. If found in a list of other strings, they represent those strings. If found in a mixed list of strings and numbers, that's an error (a single 1D AnnotatedData cannot have both strings and numbers)
- Booleans can be parsed in a numeric context (as 1 or 0).

References to other SED elements are created with a hashtag ('#') followed by a colon-delimited series of SED IDs, following the hierarchical structure of the file. If a task exports a model, that model will always be accessible with `"[task ID].model"`. Once a base type is reached,

elements of AnnotatedData may be accessed with "[index]", or by the label of the desired element, i.e. "['id']". Similarly constants can be referenced with "#constants:[constantID]".

Some examples:

- #tasks:sim1
- #tasks:sim1[0]
- #tasks:sim1.model["S1"]
- #tasks:data_import["S1"][3]

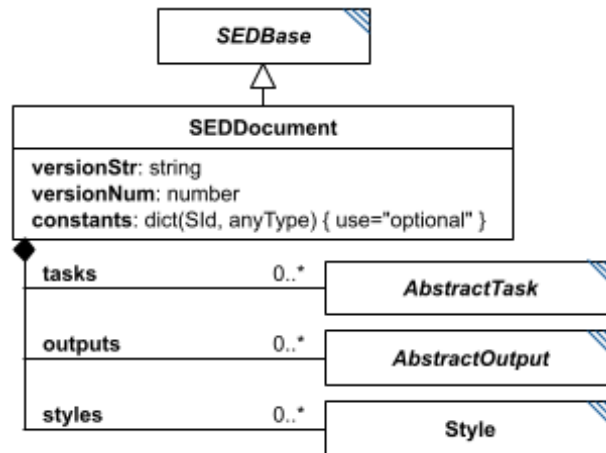
#constants:time_end

In addition, there are some types that are restricted forms of the above types:

- SId: A string serving as the identifier of the element. Usually used as the 'key' in the dictionary in which the element appears. The first character must be an English letter or the underscore character, and subsequent characters may also include digits. ('S1', '_unused_element')
- SIdRef: A string that points to another element. An SIdRef may be used absolutely anywhere, if the data type it points to is legal in its context. An SIdRef to an integer may be used instead of a '5', for example. The following rules apply:
 - The string must start with a "#"
 - The next bit of the string must be the top level element in which the element resides (i.e. '#tasks:sim1')
 - Each successive child of that top-level element must be listed, separated from the previous element with a ':' (i.e. '#tasks:repeat:subTasks:sim1')
 - If a class defines an output other than the AnnotatedData that may be accessed with its id, that element may be references with a period followed by its name. For example, if a task exports a model, you may refer to that model with '.model', i.e. '#tasks:sim1.model'.
 - If this points to AnnotatedData, you may refer to elements of that data with '[##]', with the first element in the list being 0. *[QUESTION: do we allow referencing ranges, i.e. [0:5]? Do we allow python's 'end of list' references as [-1]?]*
 - If the AnnotatedData has labels, you may instead refer to elements with '[label]'.
 - The above two methods of obtaining a subvariable may itself return a subvariable, so multi-dimensional data may be parsed this way:
"#tasks:sim1['S1'][3]"
 - If you are pointing to a model, you may refer to the numerical value of elements of that model as if it were a dictionary, and the elements are the keys (#tasks:sim1.model['S1']).
- kisaID: a string of the form "KISAO:XXXXXXX" where the X's are digits ('KISAO:0000694')
- URI: a string in URI form, as defined by w3c, i.e. 'subdir/filename.xml' or 'https://www.biomodels.org/biomodels/services/download/get-files/MODEL6615119181/4/BIOMD0000000010_url.xml'
- URN: a string in URN form, i.e. 'urn:sedml:language:sbml' or 'urn:sedml:symbol:time'

- positive integer: a number that must be an integer greater than 0.
- non-negative integer: a number that must be an integer with value 0 or greater.
- markdown: a string in the CommonMark format.
- qualifier: a string in the format "namespace:qualifier". In XML, 'namespace' is defined by a URI; in SED, several namespaces are predefined that reference those URIs that are commonly used in modeling spaces. Examples include "bqbiol:hasPart", "dc:title", and "bibo:Journal".
- ScaleType: Derived from string: either 'linear' or 'log10'.
- CurveType: Derived from string: 'points', 'bar', 'barStacked', 'horizontalBar', 'horizontalBarStacked', and 'shadedArea'.
- SurfaceType: Derived from string: 'parametricCurve', 'surfaceMesh', 'surfaceContour', 'contour', 'heatMap', 'stackedCurves', and 'bar'.

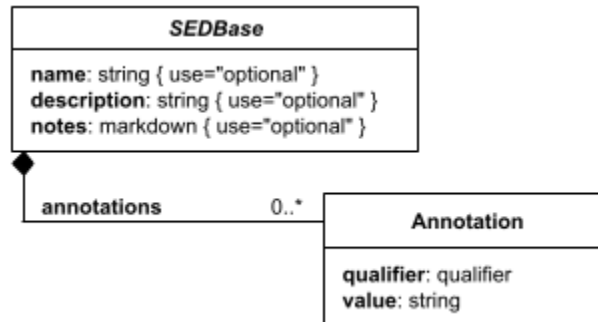
The Document



A SED 2 document has two required children (`versionStr` and `versionNum`), and four optional children: `constants`, `tasks`, `outputs`, and `styles`. The `versionStr` is a string, `versionNum` a number, and the rest are dictionaries. The `constants` child is simply a dictionary of ids associated with data, and the `tasks` and `outputs` are each dictionaries, with `tasks` containing `AbstractTask`-derived objects, and `outputs` containing `AbstractOutput`-derived objects. The keys to the dictionaries are the IDs of the elements, and the values are the objects themselves (usually dictionaries, again).

```
{
  "versionStr": "SED v2.0 alpha",
  "versionNum": 2.0,
  "constants": {
    "maxTime": 200,
    "pi": 3.14159,
    "importantSpecies": ["S1", "S2", "S3"]
  },
  "tasks": {
    "model1": {},
    "sim1": {}
  },
  "outputs": {
    "plot1": {}
  },
  "styles": {
    "style1": {}
  }
}
```

Every Element (SEDBase)

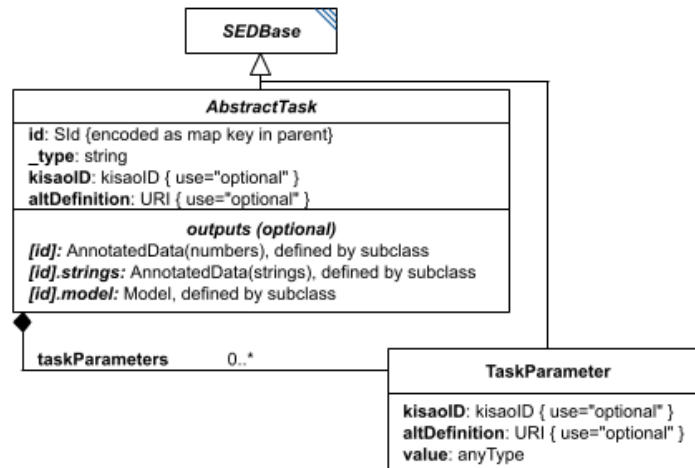


Every element in SED may have the optional key/value pairs of a name, description, notes, and annotations. The name and description are unrestricted strings; notes is markdown, and the annotations are qualifier/value pairs that describe the parent object.

Many elements of SED have an attribute labeled 'id': this is not a child of that object, but instead is the dictionary key in JSON that corresponds to that object. Anything there can be multiples of in the same list (tasks, styles, algorithm parameters, etc.) will have an id that is its key in the parent dictionary; one-off elements (ranges, mostly) will not.

```
{
  "tasks": {
    "ex1": {
      "name": "An example for the spec",
      "description": "Every element can describe what it is here.",
      "notes": "This field can have *markdown*! ###Hooray!",
      "annotations": [
        {
          "qualifier": "bibo:Journal",
          "value": "Proc. Natl. Acad. Sci. U.S.A. 1997:94(17):9147-52"
        },
        {
          "qualifier": "dc:license",
          "value": "http://creativecommons.org/publicdomain/zero/1.0/"
        }
      ]
    }
  }
}
```

AbstractTask



Every Task derives from this base *AbstractTask* class, which in turn inherits from *SEDBase*. It has an ID (used as the key in the 'tasks' dictionary), a `_type` that defines what the class of the task is, and may have a `kisaoID` and/or `altDefinition` that may further specify the task, and any number of `taskParameter` children. Each *AbstractTask* subtask will state whether the `kisaoID` is necessary, and, if so, what allowed values it may have. Each `taskParameter` child may have a `kisaoID` that defines the algorithm parameter, and the value of that parameter. If a human-readable description is needed, remember that it, too, inherits from *SEDBase*, and can therefore be given a `name` or a `description`.

The '`altDefinition`' of the *AbstractTask* and *taskParameter* is for tasks and parameters that have not been codified in KiSAO. In these cases, the `altDefinition` should be a URI that defines that task or parameter. If a future update of KiSAO includes this new task or parameter, the new ID can be substituted in without further change, with the `altDefinition` remaining as added information for readers (particularly those who have not been updated with the latest set of KiSAO definitions).

Every subclass of *AbstractTask* must define what its output or outputs are. In particular, the id of a Task (`#tasks:id`) will often be used as input for other tasks, and, if defined, must always be an *AnnotatedData* containing values (though the dimensions of that data will vary). The id of a task followed by `'.model'` (`#tasks:id.model`) must be used when the task provides a model as output. The id of a task followed by `'.strings'` (`#tasks:id.strings`) must be used when the task provides a list of strings as output. Other outputs may be used if those output types are not enough, but key reuse is important: any common id suffix should be standardized.

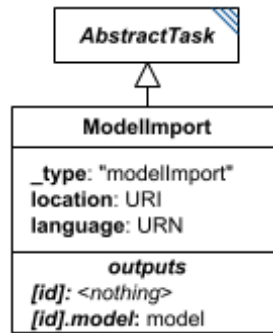
```
{
  "tasks": {
    "sim1": {
      "_type": "explicitODESimulation",
```

```
"kisaoID": "KISAO:0000694",
"taskParameters": [
  {
    "kisaoID": "KISAO:0000209",
    "name": "relative tolerance",
    "value": 1e-6
  },
  {
    "kisaoID": "KISAO:0000571",
    "name": "Absolute tolerance adjustment factor",
    "value": 1e-12
  },
  {
    "kisaoID": "KISAO:0000216",
    "name": "Integrate reduced model",
    "value": false
  }
]
}
}
```

Import Tasks

Two tasks are designed to import files from an external source: `ModelImport` and `DataImport`. All other tasks use predefined input (i.e. an attribute with a value '5'), or use information defined elsewhere in the SED document itself.

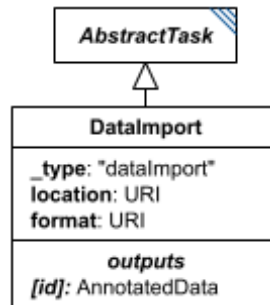
ModelImport



A `ModelImport` `location` is a URI that points to a model file. The `language` is a URN indicating what kind of model it is. Neither the `kisaoID` nor `altDefinition` attributes need be used for a `ModelImport`. The output of a `ModelImport` is the imported model, and is accessible with the id of the task appended by '.model', i.e. "`#tasks:model1.model`". The ID of the task itself does not mean anything, and may not be used.

```
{
  "tasks": {
    "model1": {
      "_type": "modelImport",
      "location": "filename.xml",
      "language": "urn:sedml:language:sbml"
    }
  }
}
```

DataImport

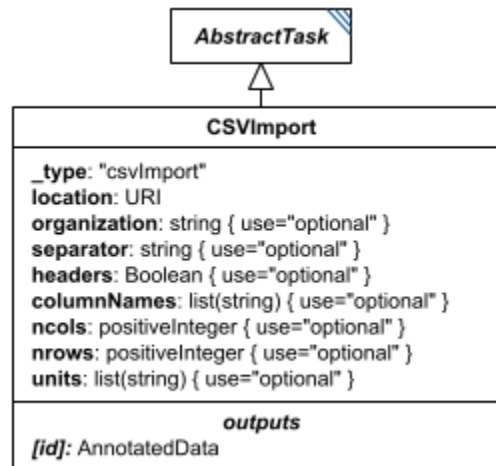


The DataImport task is designed to pull in a file at the 'location' (in the format defined in 'format'), and importing it into an AnnotatedData object. Neither the `kisaoID` nor `altDefinition` attributes need be used for a DataImport. If need be, taskParameters can be used to further specify how to import a particular file.

The DataImport task is designed to be used for non-standard file formats. Common formats such as CSV will have their own specialized importer class.

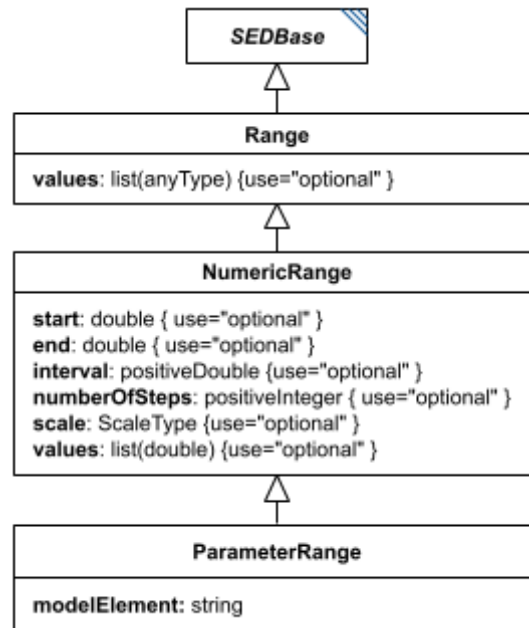
```
{
  "tasks": {
    "data1": {
      "_type": "dataImport",
      "location": "filename.csv",
      "format": "http://purl.org/NET/mediatypes/text/csv",
      "taskParameters": [
        {
          "name": "delimiter",
          "value": "\\t",
          "altDefinition":
"https://en.wikipedia.org/wiki/Delimiter-separated_values"
        },
        {
          "name": "Header row",
          "value": true,
          "altDefinition":
"https://support.ecwid.com/hc/en-us/articles/8578428241436-Format-guidelines-for-the-CSV-file#-header"
        }
      ]
    }
  }
}
```

CSVImport



The CSVImport task imports an CSV file found at 'location' and converts it to an AnnotatedData object. All of the other arguments are optional,

Range, NumericRange, ParameterRange



A Range defines a list of values to be taken in sequence. It is not typically used as a Task itself, but as the 'range' child of a Task. A base Range class simply defines a 'values' attribute that fully defines the range. The child NumericRange class restricts the 'values' attribute, if used, to be doubles, but also allows other ways to define the range.

For a NumericRange object, one way to define the range is to explicitly list them all in the **values** attribute. If this is done, no other attributes may be defined. Five other ways to define the range are to define a subset of the four attributes: **start**, **end**, **numberOfSteps**, and **interval**. One option uses exactly one of the four attributes:

- **numberOfSteps**. When only this attribute is defined, a range is defined that starts at zero and goes to numberOfSteps with an interval of 1.

The other options all use exactly three of the four attributes:

- **start, end, and numberOfSteps (plus scale)**. When these three attributes are defined, the interval between start and end is divided into numberOfSteps equal intervals, and data is collected at the start and at the end of each interval. For this set of attributes, the 'scale' attribute must also be used, to say whether the number of steps are taken on a "linear" or "log10" scale. This is the only set of attributes for which 'scale' may be defined (and, if so, it *must* be defined).
- **start, numberOfSteps, and interval**. When these three attributes are defined, numberOfSteps data points are collected every interval after the start, with an implied end.
- **end, numberOfSteps, and interval**. When these three attributes are defined, numberOfSteps data points are collected every interval up to the end, with an implied start.

- **start, interval, and end.** When these three attributes are defined, data is collected every interval past the initial start until the end is reached. When that time is reached, the end is collected as the final time point, whether or not it fell on (or near) a point evenly divisible by the interval. For example, a simulation with a start of 0, an end of 10, and an interval of 3 would produce time points at 0, 3, 6, 9, and 10.

To determine whether to include a calculated value as well as the end vs. just including the end, multiply the interval by 0.000001 (1e-6) to determine the tolerance value: if the closest calculated time point is within that tolerance of the end, only include the end. If it is further away, include both.

As an example, if the interval is 3.333, the tolerance would be $3.333 * 0.000001 = 0.000003333$. Therefore, a range with a start of 0, and end of 10, and an interval of 3.333 would produce the values [0, 3.333, 6.666, 9.999, 10], since 9.999 is greater than 0.000003333 away from 10. If the interval were instead 3.333333, the produced values would be [0, 3.333333, 6.666666, 10], since 9.999999 is less than 0.000003333333 away from 10.

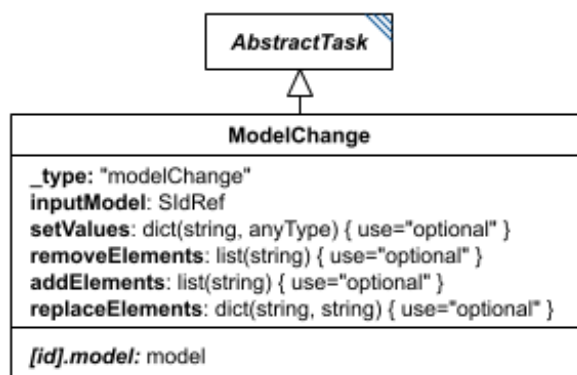
However, creating a Range where users must resort to determining the tolerance of the values to know what to output is not best practice! Output end times should generally be a simple multiple of the interval, plus the start.

A ParameterRange is for when a range of values over an element of a model is needed (such as in a ParameterScan). It defines a range of numeric values (as normal), plus the attribute 'modelElement'.

```
{
  "tasks": {
    "ex1": {
      "range": {
        "_type": "range",
        "values": [1, 2, 5, 10, 20, 50, 100]
      }
    },
    "ex2": {
      "range": {
        "_type": "range",
        "values": ["S1", "S2", "S3", "S5"]
      }
    },
    "ex3": {
      "range": {
```

```
        "_type": "numericRange",
        "start": 0,
        "end": 10,
        "numberOfSteps": 100,
        "scale": "linear"
    }
},
"ex4": {
    "range": {
        "_type": "numericRange",
        "start": 0,
        "end": 10,
        "interval": 0.01
    }
},
"ex5": {
    "range": {
        "_type": "numericRange",
        "start": 1,
        "end": 1000,
        "numberOfSteps": 3,
        "scale": "log10"
    }
},
"ex6": {
    "parameterRange": {
        "_type": "parameterRange",
        "modelElement": "S1",
        "start": "0.001",
        "end": "10",
        "numberOfSteps": 4,
        "scale": "log10"
    }
}
}
```

ModelChange



This task modifies a model (the **inputModel** SIdRef to another element in the same document) and outputs a new model with the id [id].model. There is no KiSAO term for 'modify a model', so the kisaID is not used. The four attributes that each may define changes are:

- **setValues**: the simplest of the group, this dictionary sets the string defined in the key to the corresponding value. The most common key/value pair is a model element ID as the key, and a number as the value. However, other possibilities exist, such as setting "S1.boundary" or "S1.hasOnlySubstanceUnits" to "true". The meaning of the strings and the values is determined per model format.
- **removeElements**: The string is removed from the model. In the simplest case, the element with the ID of the string is removed; one could imagine raw XML being used here, too. Again, the meaning of the strings are determined per model format. Note that if something in the model still references the removed element, it will need to be separately removed or changed. (This behavior could be changed with an taskParameter, i.e. {'removeReferencesToRemovedElements': true}).
- **addElements**: The string is added to the model. This will be, presumably, more difficult to accomplish, though 'raw XML' is again a possibility. For SBML, an Antimony-formatted string might be a good option. The meaning of the strings are determined per model format. (Also, the taskParameters may be used to refine these additions, such as 'add element 3 to spot Y in the model'.)
- **replaceElements**: Each dictionary key is removed from the model, and replaced by its corresponding value. In addition, all internal references to the key are changed to now reference its replacement instead. *[This is by far the most complicated modelChange, and is included here solely because it was part of the original SED-ML Level 1. However, the construct was rarely (if ever) used, and as such, might be a good thing to drop from SED 2. It's also worth noting that everything can already mostly be accomplished with 'add' and 'remove'. OTOH, 'where to put the added element' is at least answered with a 'replace', so maybe it's helpful after all?]*

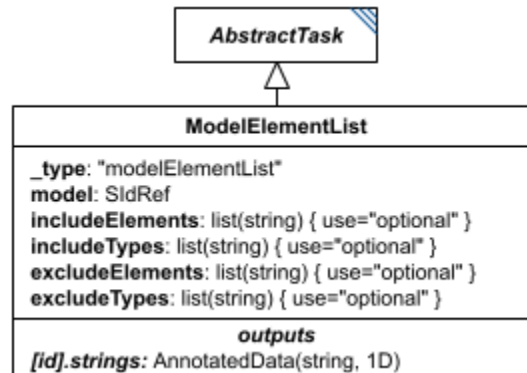
```
{
  "tasks": {
    "model1": {
      "_type": "modelChange",
```

```
    "inputModel": "#tasks:model1_orig.model",
    "setValues": {
      "S1": 5
    }
  },
  "model2": {
    "_type": "modelChange",
    "inputModel": "#tasks:model1_orig.model",
    "setValues": {
      "S1": 10,
      "S1.hasOnlySubstanceUnits": true,
      "C": 2
    },
    "removeElements": [
      "S2"
    ],
    "addElements": [
      "S_tot := S1 + S3 + S4"
    ]
  }
}
```

Lists

Some tasks are just for making lists. A `ModelElementList` creates a list of elements from a model (i.e. 'the species' or 'the nonconstant parameters'), and a `StringFormation` creates strings from other strings and numbers.

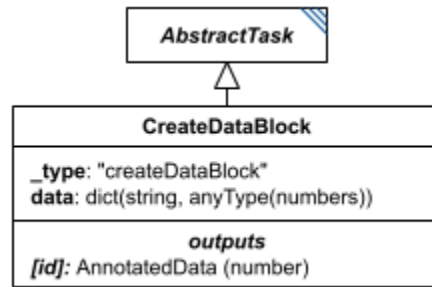
ModelElementList



The ModelElementList analyzes a model and returns a list of strings of elements of that model. They are included or excluded based on the strings in the four 'elements' and 'types' lists. All of these are lists of strings that have meaning based on the format of the referenced model. For example, in an SBML model, if you wanted a list of 'all the species plus reactions J0 and J1, but not S1', you would put 'J0' and 'J1' in the 'includeElements' list, 'species' in the 'includeTypes' list, and 'S1' in the 'excludeElements' list. This might give you the list ["S0", "S2", "S3", "S4", "J0", "J1"]. If a particular element appears in both an 'include' and an 'exclude' list, it is excluded.

```
{
  "tasks": {
    "mod1_list": {
      "_type": "modelElementList",
      "model": "#tasks:mod1.model",
      "includeElements": ["J0", "J1"],
      "includeTypes": ["species"],
      "excludeElements": ["S1"]
    }
  }
}
```

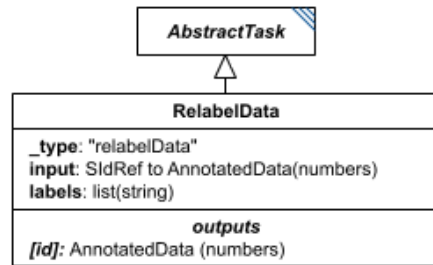
CreateDataBlock



If a new block of data needs to be created from several other blocks of data, the `CreateDataBlock` is one way to do so. The 'data' field of the task is a dictionary that links strings to numerical types (numbers, lists, `AnnotatedData`). The output of the task is a new `AnnotatedData` object with the keys as labels, and the values as content.

```
{
  "tasks": {
    "aggregate_results": {
      "_type": "createDataBlock",
      "data": {
        "S1_avg": "#tasks:repeat.aggregates['avg_sim']['S1']",
        "S1_sd": "#tasks:repeat.aggregates['sim_sd']['S1']",
        "S2_avg": "#tasks:repeat.aggregates['avg_sim']['S2']",
        "S2_sd": "#tasks:repeat.aggregates['sim_sd']['S2']",
        "S3_avg": "#tasks:repeat.aggregates['avg_sim']['S3']",
        "S3_sd": "#tasks:repeat.aggregates['sim_sd']['S3']"
      }
    }
  }
}
```

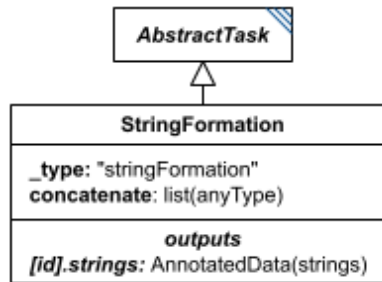
RelabelData



The RelabelData task replaces existing labels of the topmost dimension of the referenced AnnotatedData with the new labels.

```
{
  "tasks": {
    "aggregate_results": {
      "_type": "relabelData",
      "input": "#tasks:sim1",
      "labels": ["Time", "[S1]", "[S2]", "[S3]"]
    }
  }
}
```

StringFormation



For outputs, it is often useful to say things like 'this is the output line when n was equal to 3' or, more succinctly, 'n=3'. A StringFormation object takes a list of elements, converts them all to strings, then concatenates them together. If one of the elements of the list is itself a list, each individual element of that list is concatenated into a separate string, and the StringFormation returns a list of the dimensions of that element. For example, the list

`["n = ", [1, 2, 3]]`

would result in the list of strings:

`["n = 1", "n = 2", "n=3"]`

The same is true of multi-dimensional lists: the result is a multi-dimensional list of strings.

If multiple elements of the list are themselves lists, the resulting output matches corresponding elements. All list lengths must be identical. For example, the list

`[["a", "b", "c"], " = ", [1, 2, 3]]`

would result in the list of strings:

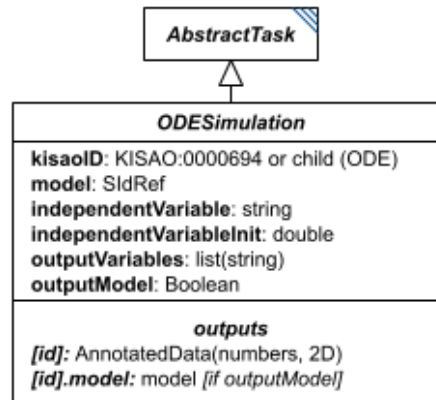
`["a = 1", "b = 2", "c=3"]`

```
{
  "tasks": {
    "n_labels": {
      "_type": "stringFormation",
      "concatenate": [ "n = ", "#tasks:loop1['range']" ]
    }
  }
}
```

Simulation Tasks

The following tasks are generally grouped under 'simulation' tasks. There is no formal definition of a 'simulation task'; they are simply tasks that take a model as input, perform some sort of analysis, and export the results of that analysis. They are grouped here for convenience.

ODESimulation



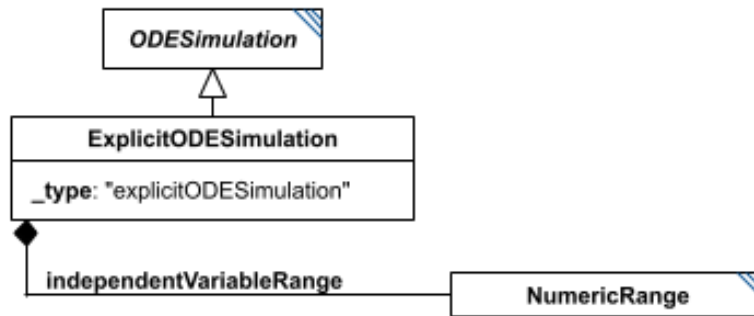
An ODESimulation is an abstract class that defines the basics of a simulation along an independent variable (such as 'time'). Its child classes further define the specifics. The kisaID must be `KISAO:0000694` (ODE solver) or a child of that value, i.e. `KISAO:0000019` (CVODE) or `KISAO:0000088` (LSODA).

The independent variable is defined by the `independentVariable` attribute, indicating what element of the model is the independent variable. In the case where the independent variable is time, but no explicit variable 'time' is present in the model, the string `"urn:sedml:symbol:time"` should be used. The `independentVariableInit` is the value of the independent variable at which the model state was determined (often `'0.0'`).

Output is determined by the `independentVariable` and the `outputVariables` attribute. It will be a 2D matrix of numbers, the first column representing the `independentVariable`, and subsequent columns representing (in order) each variable from the `outputVariables` list. These columns must be annotated with the appropriate label. Each model format will define what valid strings are, but at the very least, if there is an element in the model with an ID, that ID should be a valid string in the `outputVariables` list. In addition, the model itself will be in a new state at the end of the simulation. If that model/state is to be used as input to some other task, `'outputModel'` should be set to `'true'`. When set, `[id].model` becomes a valid model id, and may be used as inputs to other tasks or outputs.

[examples in the following sections]

ExplicitODESimulation

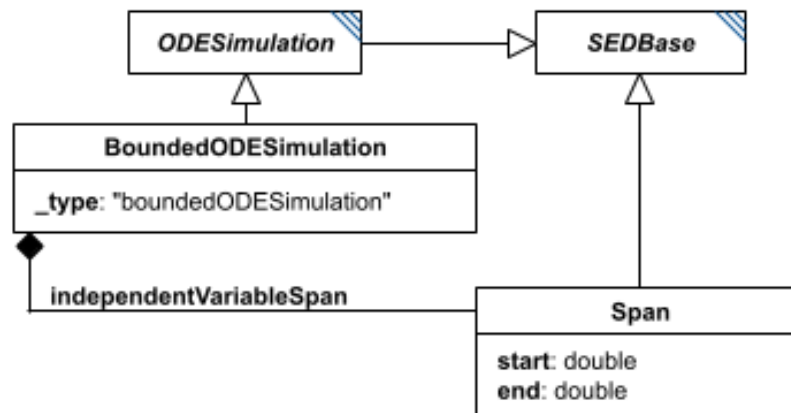


An ExplicitODESimulation simulates a model along the independentVariable with fully-defined output steps. Those points are determined by the child Range object.

Just because the range defines explicit values for output does not mean that the simulation algorithm has to work with the step sizes between those values. Usually the step size the simulator chooses will be adaptive and much smaller than the required output steps.

```
{
  "tasks": {
    "sim1": {
      "_type": "explicitODESimulation",
      "kisaoID": "KISA0:0000694",
      "model": "#tasks:model1.model",
      "independentVariable": "urn:sedml:symbol:time",
      "independentVariableInit": 0,
      "independentVariableRange": {
        "start": 0,
        "end": 20,
        "numberOfSteps": 50,
        "scale": "linear"
      },
      "outputVariables": ["S1", "S2"],
      "outputModel": false
    }
  }
}
```

BoundedODESimulation



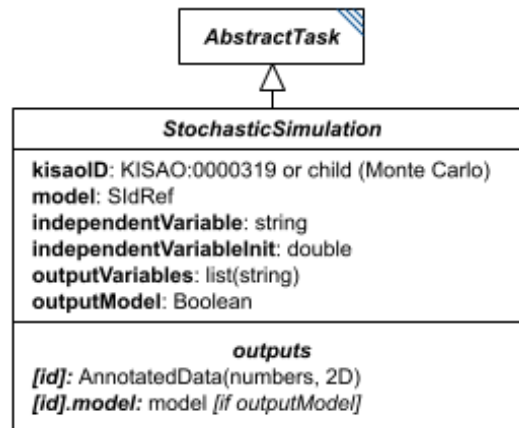
A BoundedODESimulation defines a span for the simulation it defines instead of a demarcated range: only a start and an end. Typically, the output points are more tightly packed when the values of the simulation are changing rapidly, and more spread out when the values are changing slowly. Algorithm parameters may be passed in to the solver to further refine how the output points are determined.

Span

The Span class defines a range: a start value and an end value.

```
{
  "tasks": {
    "sim1": {
      "_type": "boundedODESimulation",
      "kisaoID": "KISA0:0000694",
      "model": "#tasks:model1.model",
      "independentVariable": "urn:sedml:symbol:time",
      "independentVariableInit": 0,
      "independentVariableSpan": {
        "start": 0,
        "end": 20
      },
      "outputVariables": ["S1", "S2"],
      "outputModel": false
    }
  }
}
```

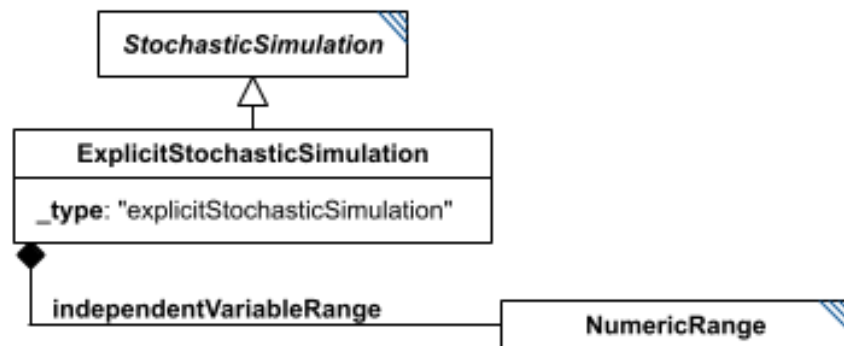
StochasticSimulation



A stochastic simulation is a simulation where the processes in the model (i.e. SBML 'reactions') are processed as statistical likelihoods of the process occurring, instead of as a continuous rate of change. This is different from a Stochastic Differential Equation, where the rates of change themselves are stochastic. Like the **ODESimulation** abstract class, a **StochasticSimulation** also has two children, one with explicit output points, and one without. Otherwise, it is identical to an **ODESimulation**, except that it performs a different analysis, and must have a **kisaID** that derives from 'KISAO:0000319' (Monte Carlo method).

Like an **ODESimulation**, output is determined by the **independentVariable** and the **outputVariables** attribute. It will be a 2D matrix of numbers, the first column representing the **independentVariable**, and subsequent columns representing (in order) each variable from the **outputVariables** list. These columns must be annotated with the appropriate label.

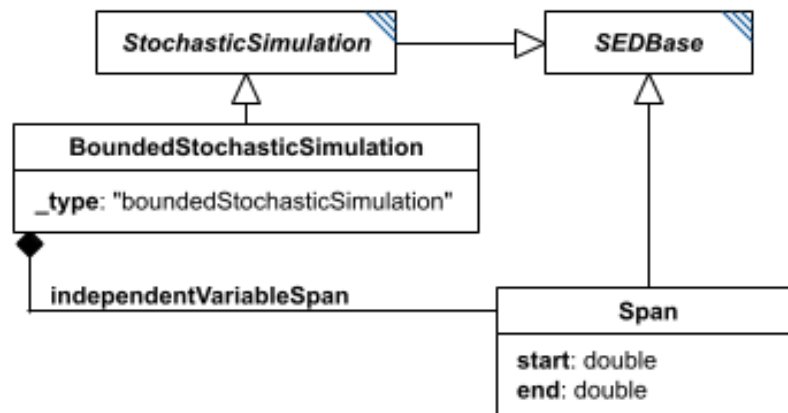
ExplicitStochasticSimulation



An ExplicitStochasticSimulation has outputs defined by its child independentVariableRange. (For a discussion on Range, see above). Note that the stochastic events in the simulation itself do not have to occur at the output points, but rather whatever values the model elements have should be recorded at the output points.

```
{
  "tasks": {
    "sim1": {
      "_type": "explicitStochasticSimulation",
      "kisaoID": "KISA0:0000319",
      "model": "#tasks:model1.model",
      "independentVariable": "urn:sedml:symbol:time",
      "independentVariableInit": 0,
      "independentVariableRange": {
        "start": 0,
        "end": 20,
        "interval": 0.05
      },
      "outputVariables": ["S1", "S2"],
      "outputModel": false
    }
  }
}
```

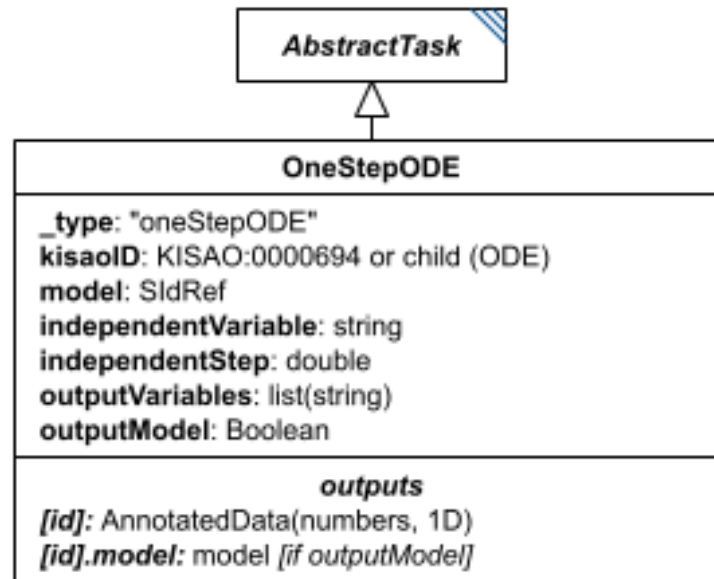
BoundedStochasticSimulation



The BoundedStochasticSimulation is a stochastic simulation where the start and end times are defined, but between those points, the simulator may output data whenever it wishes. Typically, output is generated whenever a stochastic event takes place, but in cases where this would be too frequent, the behavior can be controlled through the use of algorithm parameters.

```
{
  "tasks": {
    "sim1": {
      "_type": "boundedStochasticSimulation",
      "kisaoID": "KISA0:0000319",
      "model": "#tasks:model1.model",
      "independentVariable": "urn:sedml:symbol:time",
      "independentVariableInit": 0,
      "independentVariableSpan": {
        "start": 0,
        "end": 20
      },
      "outputVariables": ["S1", "S2"],
      "outputModel": false
    }
  }
}
```

OneStepODE

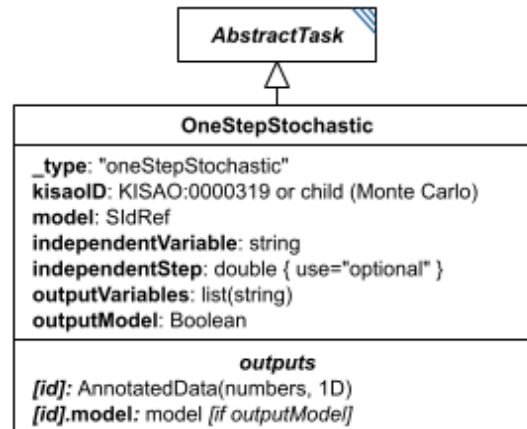


A OneStep is a simulation with exactly one output point: the values of the outputVariables after the independent variable increases by independentStep. The output is the only thing that distinguishes it from a BoundedODESimulation: internally, the simulator again may take any step size it deems appropriate, but here, the only output is the final set of points. It also is agnostic about the current value of the independent variable: it simply simulates for a given distance (the 'independentStep'), then outputs the final values for each outputVariable.

[NOTE: Is it OK that we don't know what the independent variable's value is? And that we don't output it?]

```
{
  "tasks": {
    "sim1": {
      "_type": "oneStepODE",
      "kisaoID": "KISAO:0000694",
      "model": "#tasks:model1.model",
      "independentVariable": "urn:sedml:symbol:time",
      "independentStep": 5.0,
      "outputVariables": ["S1", "S2"],
      "outputModel": false
    }
  }
}
```

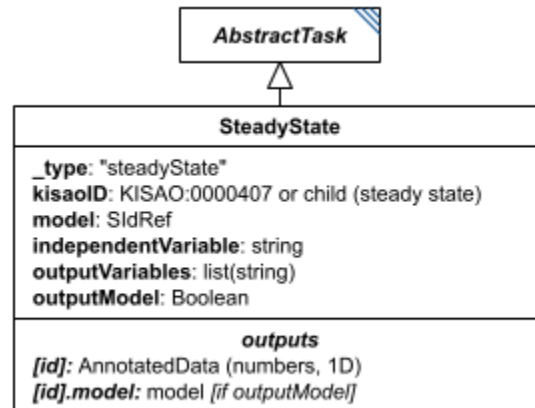
OneStepStochastic



A OneStepStochastic simulation is similar to a OneStepODE simulation, except that the independent variable step size (independentStep) may either be set or not. If it is not set, the simulation should run until a stochastic event occurs, whereupon the model changes, any outputVariables are saved, and the step size taken is saved as the [id].independentStep. If the independentStep is set, the independent variable is advanced that much, regardless of how many stochastic events occur during that interval (zero to many).

```
{
  "tasks": {
    "sim1": {
      "_type": "oneStepStochastic",
      "kisaoID": "KISAO:0000319",
      "model": "#tasks:model1.model",
      "independentVariable": "urn:sedml:symbol:time",
      "outputVariables": ["S1", "S2"],
      "outputModel": false
    }
  }
}
```

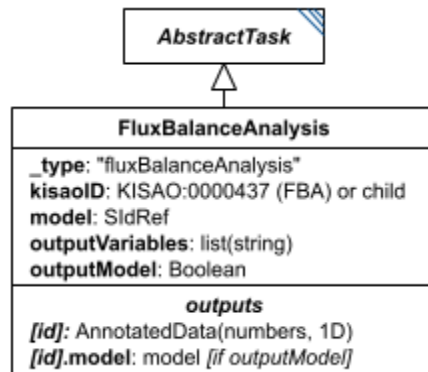
SteadyState



Calculating the steady state of a model is determining mathematically where the $\frac{dX}{dt}$ is equal to zero, when 't' is the independent variable, and 'X' is every varying element of the model. The KiSAO id must be kisaoID: KISAO:0000407 ('steady state root-finding method') or a child of that element. As for other SimulationTasks, the independentVariable is either the independent variable defined in the model, or if implicit, a URN defining that element (i.e. 'urn:sedml:symbol:time'). If the model itself is needed as an output, outputModel should be set to true, whereupon [id].model will reference that model.

```
{
  "tasks": {
    "sim1": {
      "_type": "steadyState",
      "kisaoID": "KISAO:0000407",
      "model": "#tasks:model1.model",
      "independentVariable": "urn:sedml:symbol:time",
      "outputVariables": ["S1", "S2"],
      "outputModel": false
    }
  }
}
```

Flux Balance Analysis



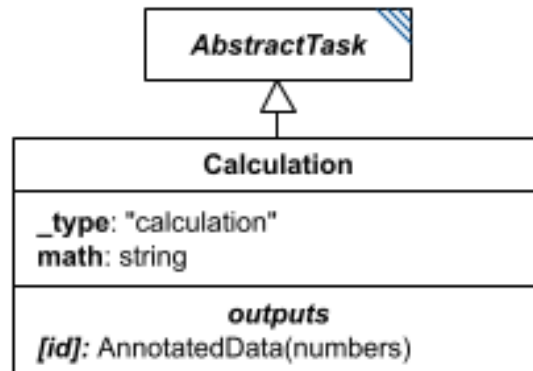
A flux balance analysis uses an objective function and reaction rate bounds (defined in the model itself) to determine the suite of reaction rates that maximize the objective function for a model. The output (much like a SteadyState analysis) is a dictionary of model variables to their final values (usually the fluxes, as those are the elements being set).

```
{
  "tasks": {
    "sim1": {
      "_type": "fluxBalanceAnalysis",
      "kisaoID": "KISAO:0000437",
      "model": "#tasks:model1.model",
      "outputVariables": ["J1", "J2", "J3"],
      "outputModel": false
    }
  }
}
```

Computational Analyses or Executions

This group of Tasks are those that perform a single mathematical task, such as performing a calculation or deriving a matrix from a model. Their outputs are generally numerical in nature.

Calculation



A Calculation performs a calculation, written in infix as the **math** attribute. Its id may then be used as the result of that calculation in other SED elements. It does not have a kisaID nor an altDefinition.

References to AnnotatedData may be used within the math infix, with the following restrictions:

- All calculations are performed on an element-by-element basis.
- If a lower-dimension element is combined with a higher-dimension element, the lower dimension is applied to each of the higher-dimension parts individually.
 - For example, in the expression "5 + [list of numbers]", the result is a list of numbers that each are five larger than they were. In the expression "[1D data] * [2D data]", the result is 2D data, where each dictionary with the same keys, where each number of the original list of numbers has been multiplied by the corresponding number in the new list.
- Combining elements of the same dimension must also have the same sizes.
 - Two added dictionaries must have the same keys.
 - Two added lists must have the same lengths.

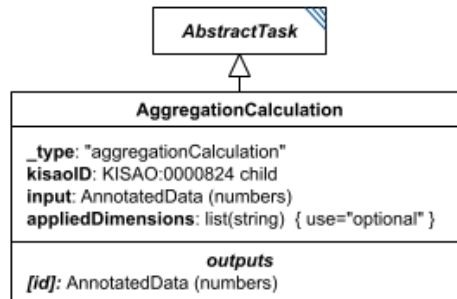
Allowed operations:

- The operands +, -, *, /, ^
- Parentheses
- PEMDAS order of operations
- The functions allowed in SBML
- The constants allowed in SBML (pi, exponentiale, etc.)

```
{
  "tasks": {
    "sim2_exp_compare": {
      "_type": "calculation",
      "math": "(#tasks:sim2 - #tasks:experiment1)^2"
    },
    "compare_summary": {
      "_type": "calculation",
```

```
    "math": "[sum(#tasks:sim1_sim2_compare['S1']) +  
sum(#tasks:sim1_sim2_compare['S2']), sum(#tasks:sim1_exp_compare['S1']) +  
sum(#tasks:sim1_exp_compare['S2']), sum(#tasks:sim2_exp_compare['S1']) +  
sum(#tasks:sim2_exp_compare['S2'])]"  
    }  
  }  
}
```

AggregationCalculation

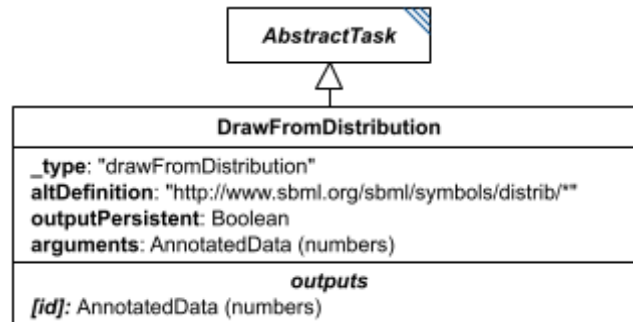


An AggregationCalculation takes dimensional data and reduces it by calculating (i.e.) the sum, average, standard deviation, or some other aggregation calculation. Which calculation is performed is determined by the kisaoID, which must be a child of 824, 'aggregation function'. If the data has multiple dimensions, only the outermost one is reduced by default, (i.e. taking the average of every value in a matrix), but the aggregation may be applied to one or more particular dimensions by using the 'appliedDimensions' child listing the ID of the dimension or dimensions to apply.

In addition, if the AggregationCalculation is being used as a Loop's aggregateOutputVariable, by default it is reduced by the dimension of that loop. For example, if a loop has a child ExplicitStochasticSimulation subTask, if that simulation's id is the 'input', the simulations are averaged together by default. This behavior may be changed by using the appliedDimensions list.

```
{
  "tasks": {
    "data1": {
      "_type": "aggregationCalculation",
      "kisaoID": "KISA0:0000849",
      "name": "cumulativeSum",
      "input": "#tasks:sim1.model['S1']"
    }
  }
}
```

DrawFromDistribution



A **DrawFromDistribution** defines a draw from one of the distributions defined in the SBML specification, namely: normal, cauchy, chisquare, exponential, gamma, laplace, lognormal, rayleigh, binomial, and poisson. The arguments to the function are defined in the arguments attribute, and the argument order is taken from the SBML definition. As an example, the arguments for a 'normal' distribution are given as '(mean, stdev)' or '(mean, stdev, min, max)'.

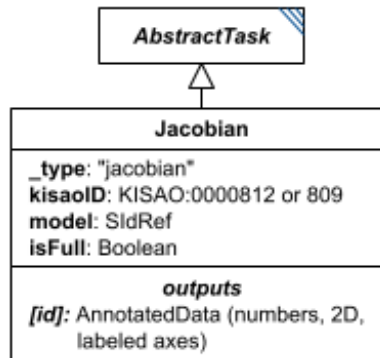
For distributions not covered by the SBML specification, other ontologies or standard definitions may be consulted (such as UncertML—that project has been dormant for over a decade, but their definitions are still pertinent).

The output is officially **AnnotatedData**, but will usually be a single number, accessible with `"id[0]"`. The type is potentially multidimensional to accommodate future draws from distributions that are for correlated values.

The **outputPersistent** attribute is used to determine whether every reference to this ID should always get the same result (true), or if every reference to this element's ID should get a new draw from the same distribution (false). Set this to 'true' if multiple tasks need to have the same draw, and set it to 'false' if multiple tasks need to have unique draws from the same distribution.

```
{
  "tasks": {
    "S1_plus_noise": {
      "_type": "drawFromDistribution",
      "altDefinition": "http://www.sbml.org/sbml/symbols/distrib/normal",
      "arguments": ["#tasks:sim1.model['S1']", 0.3],
      "outputPersistent": false
    }
  }
}
```

Jacobian

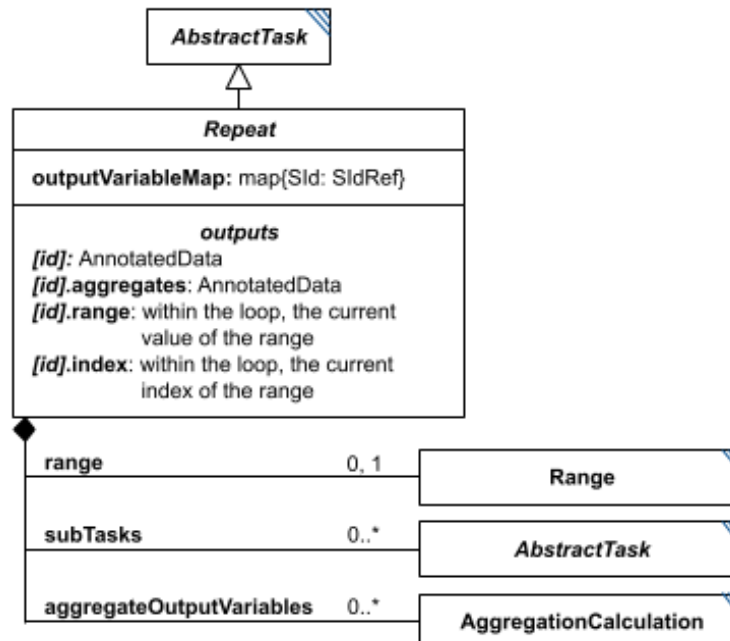


This Task calculates the jacobian of a model, a species by species matrix. The id is that matrix as a list of lists, with axes labels being the ordered list of species for the matrix. For a Jacobian, those lists are typically identical.

Whether the full or reduced matrix is returned is determined by the 'isFull' attribute (and the kisaoID).

```
{
  "tasks": {
    "mod1_jacobian": {
      "_type": "jacobian",
      "kisaoID": "KISAO:0000812",
      "model": "#tasks:mod1.model",
      "isFull": true
    }
  }
}
```

Repeat



Just like the RepeatedTask in SEDML Level 1, SED 2 needs to be able to repeat things multiple times. The subTasks are actual children of the parent Repeat, instead of references. Those subTasks may be connected to each other through inputs and outputs, and their inputs may be higher-level Tasks, but their outputs may only be referenced by the Repeat itself.

The Repeat class is abstract, as there are two types of Repeats in SED2: 'Scatter' repeats where every loop may be performed independently of any other loop, and 'Loop' repeats where the inputs of loop N depend on the outputs of loop N-1.

The 'range' child defines the loop: one loop for each element of the range.

The 'subTasks' children are the tasks to be executed during the loop. They each may take as inputs:

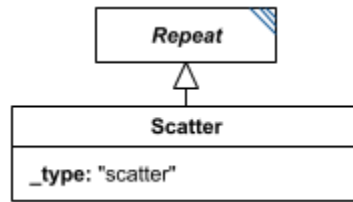
- The output of any higher-level task, constant, or style from the document.
- The output of any sibling subTask (which should be listed before it, to ensure scriptability)
- The 'range' or 'index' of the containing Repeat, which will be the particular value of the range and index for the current Repeat.

The OutputVariableMap defines what values are present in the AnnotatedData output of the Repeat's output id. The first column of the output data are the range values, and subsequent columns are labeled with the id of each outputVariable, with the content being the values referenced in the subTaskOutput for that outputVariable. If the OutputVariableMap is empty, the output of the id of the task will only contain the range values.

The 'aggregateOutputVariables' define what values are present in the AnnotatedData output of the Repeat's output id.aggregates. These children are present exclusively for the efficient storage of large amounts of data: if some Repeats loop a very large number of times, and the outputs they collect are similarly large (such as 100,000 repeats of a stochastic simulation), often only summary statistics are needed for final output—the individual values of every run are not. In these cases, instead of referencing the individual subTask outputs in an OutputVariable and subsequently calculating (for example) the mean and standard deviation of the output of those runs, they may instead be collected as 'running totals' as each loop finishes. A restriction on the individual aggregateOutputVariables: none may define an 'appliedDimensions' child (the applied dimension must be 'the Repeat'), and the 'input' must be an output value of one of the subTasks.

The 'range' and 'index' outputs may not be used anywhere but in a child subTask. Those values are available as the first column of data in the Repeat's 'id' output.

Scatter

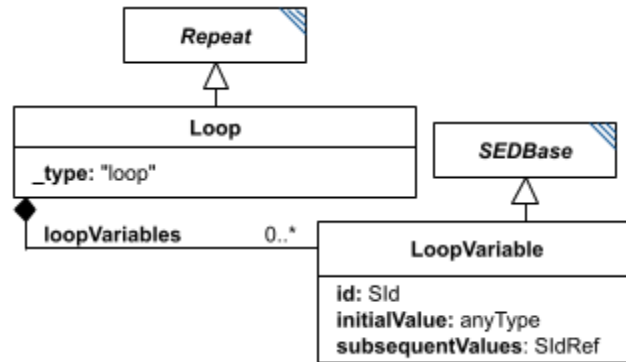


The Scatter class is just a Repeat, but with a guarantee: each repeated execution of the subTasks must be completely independent of any other repeated execution. The repeats may therefore be executed in parallel if desired.

```
{
  "tasks": {
    "repeat": {
      "_type": "scatter",
      "range": {
        "start": 0,
        "end": 10,
        "interval": 1
      },
      "aggregateOutputVariables": {
        "avg_sim": {
          "kisaoID": "KISA0:0000841",
          "name": "mean",
          "input": "#tasks:repeat:subTasks:sim1"
        },
        "sim_sd": {
          "kisaoID": "KISA0:0000842",
          "name": "standard deviation",
          "input": "#tasks:repeat:subTasks:sim1"
        }
      },
      "subTasks": {
        "sim1": {
          "_type": "explicitStochasticSimulation",
          "kisaoID": "KISA0:0000319",
          "model": "#tasks:mod1.model",
          "outputModel": false,
          "outputVariables": ["S1", "S2", "S3"],
          "independentVariable": "urn:sedml:symbol:time",
```

```
    "independentVariableRange": {
      "start": 0,
      "end": 20,
      "numberOfSteps": 50
    }
  },
  "outputVariableMap": {},
  "notes": "The only outputs from this task are the aggregateOutputVariables.
They may be accumulated as they come in, therefore, instead of calculated after
the fact."
}
```

Loop

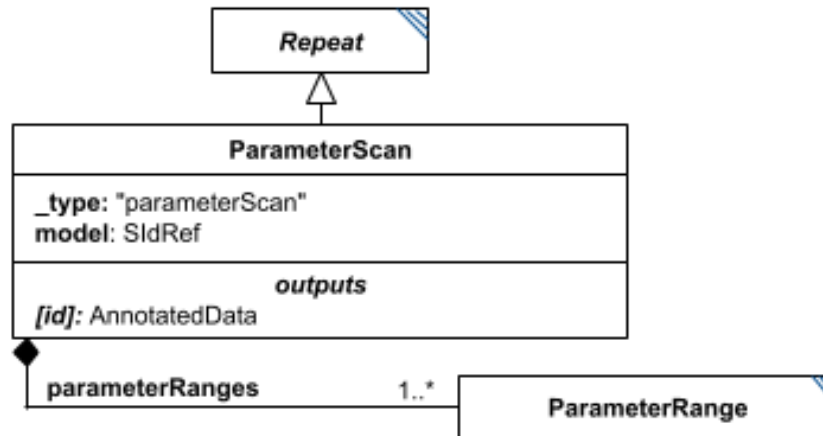


In a Loop class, each repeat depends on an output of the previous repeat. This is accomplished through the use of LoopVariables: a variable whose value is equal to its `initialValue` (sometimes referred to as a 'seed' in workflow languages) the first time through the loop, and subsequently by the referenced 'subsequentValues', which must be an output variable of one of the Loop's subTasks. When used as input in a subTask, the reference should be in the form `"#tasks:[loop_id]:loopVariables:[loopvar_id]"`

```
{
  "tasks": {
    "repeat": {
      "_type": "loop",
      "range": {
        "start": 0,
        "end": 10,
        "interval": 1
      },
    },
    "loopVariables": {
      "looped_model": {
        "initialValue": "#tasks:mod1.model",
        "subsequentValues": "#tasks:repeat:subTasks:sim1.model"
      }
    },
  },
  "subTasks": {
    "sim1": {
      "_type": "oneStepODE",
      "kisaoID": "KISA0:0000694",
      "model": "#tasks:repeat:loopVariables:looped_model",
      "outputModel": true,
      "outputVariables": ["S1", "S2", "S3"],
    },
  },
}
```

```
    "independentVariable": "urn:sedml:symbol:time",  
    "independentStep": 1.0  
  },  
},  
"outputVariableMap": {  
  "sim1_repeat": "#tasks:repeat:subTasks:sim1"  
}  
}  
}  
}
```

ParameterScan



A parameter scan is when simulations or analyses are performed on several different versions of a model, where some element or elements of that model are systematically varied over a range. The dimensions of the output are the number of elements being scanned. The ParameterScan class is a child of Repeat that adds two things: a 'model' child referencing the model to be scanned, and one or more ParameterRange children, each of which are to be scanned. The subTasks are the same, though one of them must reference the 'model' child of the ParameterScan ('#tasks:[scanid]:model') as its input. The output is also the same: outputVariables and/or aggregateOutputVariables may be defined to declare what data to collect from the scan.

Each of ParameterRange children must have a different modelElement defined, each of which must be the id of an element of the child model. This element will then be scanned over the range of values defined.

Any number of ParameterRange children may be added to the ParameterScan, with each one adding a new dimension to the output. The model object will be initialized within the subTasks to have the values of each value of every range in every combination.

The rest of the mechanics are the same as a normal Scatter (and, in fact, could be duplicated with nested Scatter objects along with careful use of ModelChange tasks).

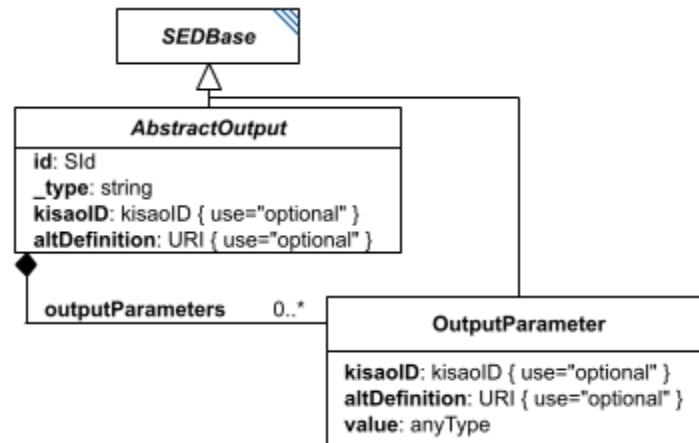
```
{
  "tasks": {
    "scan_S1_and_k2": {
      "_type": "parameterScan",
      "model": "#tasks:mod1.model",
      "parameterRanges": [
        {
          "modelElement": "S1",
```

```

        "start": 0,
        "end": 10,
        "interval": 1
    },
    {
        "modelElement": "k2",
        "start": 0.0001,
        "end": 10,
        "numberOfSteps": 5,
        "scale": "log10"
    }
],
"subTasks": {
    "sim1": {
        "_type": "explicitODESimulation",
        "kisaoID": "KISA0:0000694",
        "model": "#tasks:scan_S1_and_k2:model",
        "outputModel": false,
        "outputVariables": ["S1", "S2", "S3"],
        "independentVariable": "urn:sedml:symbol:time",
        "independentVariableRange": {
            "start": 0,
            "end": 20,
            "interval": 0.05
        }
    }
},
"outputVariableMap": {
    "sim1_scan": "#tasks:scan_S1_and_k2:subTasks:sim1"
},
"notes": "The simulation data from subTasks:sim1 is exported as 2D data for
S1 values by k1 values."
}
}
}

```

AbstractOutput

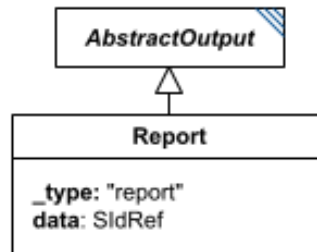


AbstractOutputs take data from the Tasks and package it for the user. This can take two basic forms: the report (just text and numbers) or the plot (graphics). If an output is needed that is not defined in SED2, the kisaolD or altDefinition children may be used to define the algorithm, along with any taskParameter children necessary to define the algorithm's inputs.

Any AbstractOutput object may not be used as input for anything else in SED2: it is the final stage of processing. If something needs to be used multiple times, it should be created in a Task.

Any output does not dictate the form it takes: they may be used within a program, and simply be an object containing the data. It may be used on a web page, and display the results for the user. Or it may be used on a desktop, and produce files. Just as SED2 does not dictate what specific program executes a task, merely what that task must accomplish, it also does not dictate how to bundle or export the results, merely what those results must contain. *[This would be a good time to mention the 'Wednesday file', which contains information about how a particular SED2 run was substantiated.]*

Report

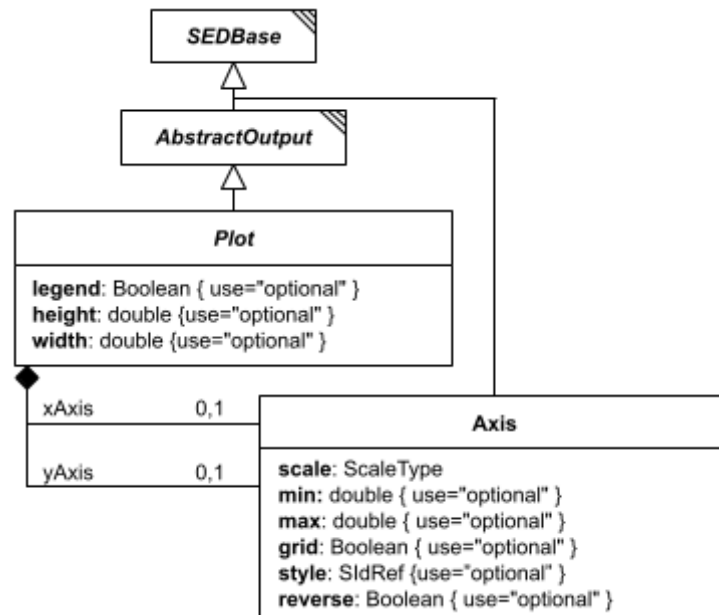


A report exports data: an AnnotatedData object from . The simplest form a report can have is of two-dimensional data: in that format, it can be displayed easily to the user, or saved to a simple CSV file. However, some data will simply have higher dimensionality. In those cases, it may be more useful to export to an HDF5 file, or some other format designed to contain multidimensional data. This is beyond the scope of SED2: the format merely dictates what data should be included, not how it is to be stored.

The data must be a reference to AnnotatedData, from any task in the SED2 document, or to a slice from an AnnotatedData. If the referenced data has labels, those labels should be exported (i.e. in a CSV file, they should be used as row or column labels).

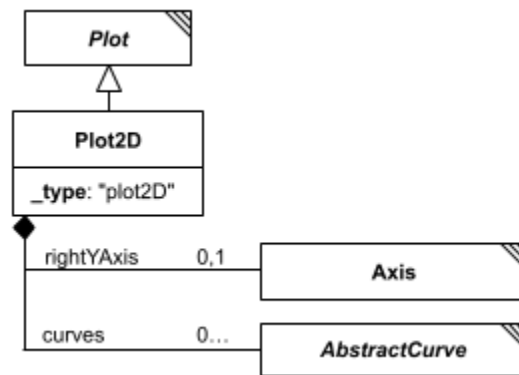
```
{
  "outputs": {
    "sim1Report": {
      "_type": "report",
      "data": "#tasks:sim1"
    },
    "sim2TruncatedReport": {
      "_type": "report",
      "data": "#tasks:sim2['S1']"
    }
  }
}
```

Plot



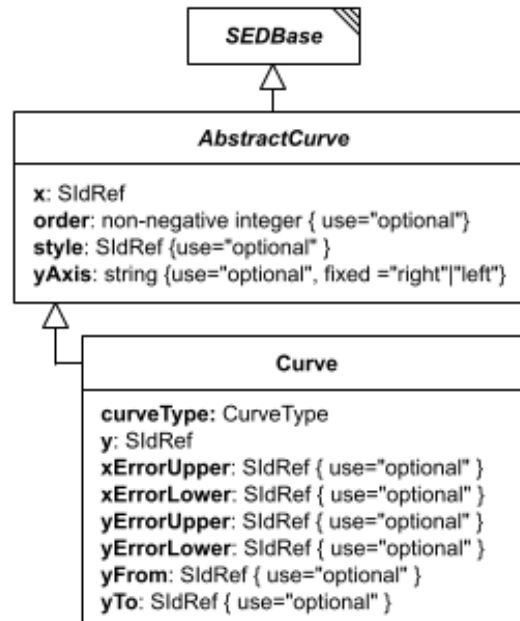
A Plot is the abstract parent of the Plot2D and the Plot3D. It defines the x and y axes, the size, and whether or not there should be a legend.

Plot2D



A **Plot2D** is a plot of two-dimensional data. It adds a possible right Y axis, and the curves that should be shown. [Same as *SED-ML*]

AbstractCurve

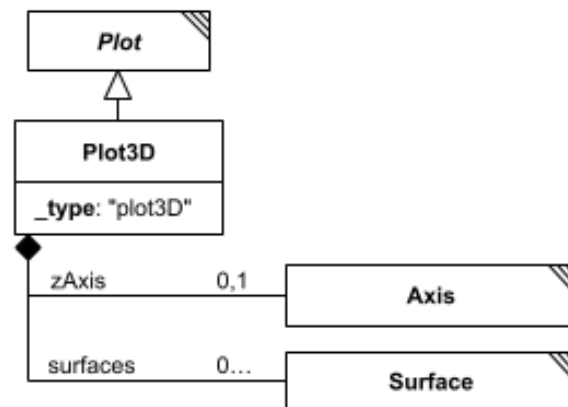


The AbstractCurve class defines the basics of a curve. note that 'yFrom' and 'yTo' are only valid when the curveType is 'shadedArea'. [Same as SED-ML]

```
{
  "outputs": {
    "Fig1": {
      "_type": "plot2D",
      "name": "Figure 1",
      "xAxis": {
        "name": "Time",
        "scale": "linear"
      },
      "yAxis": {
        "name": "Concentrations",
        "scale": "linear"
      },
      "curves": {
        "Sim1_S1": {
          "curveType": "points",
          "x": "#tasks:sim1['time']",
          "y": "#tasks:sim1['S1']"
        },
        "Sim1_S2": {
          "curveType": "points",
```

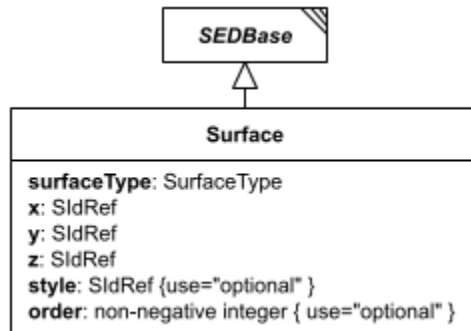
```
      "x": "#tasks:sim1['time']",
      "y": "#tasks:sim1['S2']"
    }
  }
}
}
```

Plot3D



A Plot3D is a plot of three-dimensional data. It adds the Z axis, and defines what surfaces should be shown. [*Same as SED-ML*]

Surface



A surface defines a 3D surface to plot on a Plot3D. [Same as SED-ML]

```
{
  "outputs": {
    "Fig1": {
      "_type": "plot3D",
      "name": "Figure 1",
      "xAxis": {
        "name": "k1",
        "scale": "linear"
      },
      "yAxis": {
        "name": "k2",
        "scale": "log10"
      },
      "zAxis": {
        "name": "Concentrations",
        "scale": "linear"
      },
      "surfaces": {
        "S1": {
          "surfaceType": "parametricCurve",
          "x": "#tasks:scan_k1_and_k2['k1']",
          "y": "#tasks:scan_k1_and_k2['k2']",
          "z": "#tasks:scan_k1_and_k2['S1']"
        },
        "S2": {
          "surfaceType": "parametricCurve",
          "x": "#tasks:scan_k1_and_k2['k1']",
          "y": "#tasks:scan_k1_and_k2['k2']"
```

```
        "z": "#tasks:scan_k1_and_k2['S2']"  
    }  
}  
}  
}
```

Style

[To be filled in; should be a straight copy of SED-ML's Style class.]